Boost.Decimal: A Portable, Header-Only, constexpr and GPU-Capable Implementation of IEEE 754 Decimal Floating-Point Arithmetic

MATT BORLAND, The C++ Alliance, USA
CHRISTOPHER KORMANYOS, Independent Researcher, Germany

Decimal floating-point arithmetic, standardized in IEEE 754, represents values such as 0.1 exactly and is required wherever results must match base-ten expectations, most notably in finance and commerce. The two established implementations, the Intel Decimal Floating-Point Math Library and IBM decNumber, are C libraries: they are not header-only, expose no C++ compile-time interface, do not run on accelerators, and do not cover the full range of CPU architectures used today. We present Boost.Decimal, a header-only C++14 implementation of the IEEE 754 decimal32, decimal64, and decimal128 types (and three non-conforming variants tuned for runtime speed) that requires no external dependency, evaluates at compile time through constexpr, executes on CPUs and on NVIDIA GPUs from a single source, and is validated for cross-platform bit-for-bit consistency across x86, ARM, s390x, PPC64LE, and bare-metal targets. We describe the design, the arithmetic algorithms, the conformance and reproducibility testing, and a performance study showing that Boost.Decimal matches or exceeds the Intel reference library across the basic operations on several platforms, for example computing decimal64 additions several times faster than the Intel reference on x86-64. Because the types satisfy the concept requirements of generic C++ numerical code, existing libraries such as Boost.Math operate on decimal data without modification, making Boost.Decimal a basis for reproducible decimal analysis. The library is available at https://github.com/boostorg/decimal.

CCS Concepts: • **Mathematics of computing** → **Mathematical software**; *Numerical analysis*; • **Software and its engineering** → *Software libraries and repositories*; • **Computing methodologies** → Parallel computing methodologies.

Additional Key Words and Phrases: decimal floating-point, IEEE 754, decimal32, decimal64, decimal128, binary integer decimal, BID, C++, constexpr, header-only library, GPU, CUDA, portable numerical software, reproducibility, Boost

ACM Reference Format:
Matt Borland and Christopher Kormanyos. 2026. Boost.Decimal: A Portable, Header-Only, constexpr and GPU-Capable Implementation of IEEE 754 Decimal Floating-Point Arithmetic. *ACM Trans. Math. Softw.* 0, 0, Article 0 (2026), 18 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The binary floating-point types that dominate scientific computing store the significand in base two. As a consequence they cannot represent most finite base-ten fractions exactly: the value 0.1 has no finite binary expansion, and the familiar evaluation of `0.1 + 0.2` in IEEE 754 binary64 yields

Authors' Contact Information: Matt Borland, The C++ Alliance, USA, matt@mattborland.com; Christopher Kormanyos, Independent Researcher, Germany, e_float@yahoo.com.

ACM 1557-7295/2026/0-ART0
https://doi.org/XXXXXXX.XXXXXXX

0.30000000000000004 rather than 0.3.[1] The error is small, but it is present before any arithmetic is performed, because it enters when a base-ten literal or an input string such as a price is rounded to the nearest binary value. In domains where a computed result must equal the value a person obtains by hand, in particular finance, accounting, tax and currency calculations, and commerce, this discrepancy is unacceptable, and it is the reason the 2008 revision of IEEE 754 standardized decimal floating-point arithmetic [6, 12].

Listing 1. Binary `double` rounds the literals 0.1 and 0.2; the decimal type represents them exactly.

```
using namespace boost::decimal::literals;
std::cout << 0.1 + 0.2 << '\n';        // double:      0.30000000000000004
std::cout << 0.1_DD + 0.2_DD << '\n';  // decimal64_t: 0.3
```

Decimal floating-point types store the significand in base ten, so a value such as 0.1 or 0.3 is represented exactly, and they offer a far wider range than the fixed-point representations that are sometimes used to avoid binary rounding. Despite the standard being more than fifteen years old, decimal arithmetic remains awkward to use from modern C++. The two established implementations, the Intel Decimal Floating-Point Math Library [5, 14] and IBM's decNumber [7], are C libraries. They must be compiled and linked rather than simply included, they expose no compile-time (`constexpr`) interface, they do not execute on graphics processors, and neither covers the full range of CPU architectures in use today. The C++ standards proposals that would have added decimal types [15, 17, 20] planned to wrap these C libraries and left `constexpr` support optional, and none has been adopted.

This article presents Boost.Decimal, a from-scratch implementation of IEEE 754 decimal floating-point arithmetic that closes this gap. It provides the three standard types `decimal32_t`, `decimal64_t`, and `decimal128_t` (7, 16, and 34 significant decimal digits), together with three companion types that relax the storage layout for greater runtime speed. The library is part of the Boost C++ Libraries collection, distributed since release 1.91.[2] Its contributions are:

(1) A header-only, dependency-free implementation that requires only C++14 and builds with toolchains as old as `clang++ 6`, removing the integration friction of the existing C libraries (Section 3).
(2) Full `constexpr` evaluation: every operation, including parsing, formatting, and the elementary functions, can be evaluated at compile time (Sections 3 and 4).
(3) Execution on NVIDIA GPUs through CUDA, from the same source as the CPU path. To our knowledge this is the first decimal floating-point library with GPU support (Section 7).
(4) Verified bit-for-bit identical results across a wide matrix of operating systems and architectures, including big-endian s390x and bare-metal ARM Cortex-M (Sections 6 and 8).
(5) A performance study showing that Boost.Decimal matches or exceeds the Intel reference library across the basic operations, and is faster than the GCC `_Decimal` built-in types in many of them (Section 9).
(6) Composition with the existing C++ numerical ecosystem: because the types model the concepts that generic numerical code expects, libraries such as Boost.Math operate on decimal data unchanged, putting a broad range of analysis within reach on exact decimal values (Section 10).

We are deliberate about what is and is not new. The decimal formats, the binary integer significand (BID) encoding, and conformance to IEEE 754 are established prior art, shared with the Intel and IBM libraries. The contribution of this work is the combination that no prior implementation offers:

[1] https://0.30000000000000004.com
[2] https://github.com/boostorg/decimal

a single portable, header-only, `constexpr` source that runs conformantly on CPUs, GPUs, and bare-metal targets while matching the performance of the reference software.

2 Background and Related Work

2.1 IEEE 754 decimal floating-point

The 2008 revision of IEEE 754 [12], retained in the 2019 revision [13], defines three interchange formats for decimal floating-point: decimal32, decimal64, and decimal128, providing 7, 16, and 34 significant decimal digits respectively. A finite value is a signed integer significand scaled by a power of ten. The standard specifies the arithmetic operations, the rounding modes (with round to nearest, ties to even as the default), the special values (signed zeros, infinities, and quiet and signaling NaNs), and the behavior of conversions, following the General Decimal Arithmetic specification of Cowlishaw [6].

A property unique to decimal is the cohort. Because the significand and exponent are stored separately, one numerical value has several representations: for example 1e5, 10e4, and 100e3 all denote one hundred thousand. Binary formats have no cohorts. Preserving or normalizing cohorts is a design concern that does not arise for binary types, and we return to it when discussing input and output (Section 5).

2.2 Encodings: BID and DPD

IEEE 754 permits two encodings of the significand field. The Binary Integer Decimal (BID) encoding stores the significand as a binary integer and is intended for software implementations; the Densely Packed Decimal (DPD) encoding packs three decimal digits into ten bits and is intended for hardware. Boost.Decimal stores values in BID internally for the conforming types, and provides conversions to and from both BID and DPD bit strings, so values can be exchanged with hardware decimal units and with other implementations.

2.3 Existing implementations

The reference software implementation of decimal arithmetic is the Intel Decimal Floating-Point Math Library, which Cornea et al. introduced as the first software implementation of IEEE 754 decimal using the BID encoding [5]; it is the de facto industry standard [14]. IBM's decNumber is a portable ANSI C reference implementation of General Decimal Arithmetic and is used inside GCC and the International Components for Unicode [7]. GCC also ships libdecnumber and a copy of the Intel BID library to support its `_Decimal32`, `_Decimal64`, and `_Decimal128` extension types. All of these are C libraries: they are compiled and linked, expose no `constexpr` interface, and do not run on accelerators.

On the C++ side, ISO/IEC TR 24733 [15] and the later WG21 proposals N3407 and N3871 [17, 20] specified decimal class types for C++, but proposed to implement them by wrapping the existing Intel or IBM C libraries and treated `constexpr` support as allowed rather than required; no proposal has been adopted into the standard. Portable header-only numerical libraries in the broader Boost and TOMS tradition, such as the e_float multiple-precision system [19], MPFR [9], and the customized floating-point framework FloatX [8], demonstrate the value of a header-only, standards-integrated C++ approach, but none provides IEEE 754 decimal floating-point.

2.4 Decimal on GPUs

There is, to our knowledge, no prior IEEE 754 decimal floating-point implementation that runs on a graphics processor. The closest existing work is the fixed-point decimal support in NVIDIA's RAPIDS cuDF library, whose `decimal32`, `decimal64`, and `decimal128` column types are fixed-point

Table 1. Algorithmic foundations: the established methods the implementation uses, by component. The novelty of this work is their unified, portable, `constexpr` delivery, not the methods themselves.

Component	Method	Reference
Wide multiplication	Schoolbook (Algorithm M)	[18]
Wide division	Algorithm D; reciprocal 3/2	[18, 21]
Divide or round by 10^k	Reciprocal by invariant divisor	[10, 25]
Integer to text	JEAIII	[2]
Shortest float to text	Ryu	[1]
Square root	Table plus Newton (SoftFloat)	[11]
Range reduction	Cody-Waite	[4]
Polynomial approximation	Minimax (Remez)	[22]
Elliptic integrals	Arithmetic-geometric mean	[19, 24]
Riemann zeta	Accelerated alternating series	[3]
Series summation	Kahan compensated sum	[16]

Table 2. Boost.Decimal compared with the established decimal floating-point software. Each property has precedent individually; providing all of them together is the contribution of this work.

	Boost.Decimal	**Intel BID**	**IBM decNumber**	**GCC `_Decimal`**
Language	C++14	C	C	C extension
Header-only	yes	no	no	no
`constexpr`	yes	no	no	no
GPU / CUDA	yes	no	no	no
Architectures	x86/ARM/s390x/PPC64LE	x86 only	portable C	GCC targets
License	BSL-1.0	BSD-style	ICU	GPL + RLE

(a runtime scale, in the manner of Apache Spark and Java BigDecimal), not floating-point, and whose arithmetic manipulates scales rather than a self-adjusting exponent [23]. The IEEE 754 support in NVIDIA GPU hardware is binary only. Decimal floating-point on the GPU is therefore new ground, which we describe in Section 7.

2.5 Algorithmic foundations

Boost.Decimal does not invent new arithmetic; it assembles well-established algorithms in a single portable, `constexpr` C++ form, and cites each where it is used. The wide-integer multiply and divide underneath the significand arithmetic are the schoolbook Algorithm M and Algorithm D of Knuth [18], with the inner quotient step using the reciprocal method of Moller and Granlund [21]. Division and rounding by powers of ten use reciprocal multiplication by an invariant divisor [10, 25]. Integer-to-text conversion uses the JEAIII method [2], and shortest round-trip output uses Ryu [1]. The square root follows the table-and-Newton approach of Berkeley SoftFloat [11], adapted to base ten. The elementary functions combine Cody-Waite range reduction [4], minimax polynomial approximations [22], the arithmetic-geometric mean for elliptic integrals [19, 24], an accelerated series for the Riemann zeta function [3], and Kahan compensated summation [16]. Table 1 collects these. The contribution of this work is not the algorithms individually but their delivery together as one portable, header-only type.

2.6 Summary of the gap

Table 2 summarizes how Boost.Decimal differs from the established implementations. Each individual property has precedent; the contribution of this work is to provide all of them at once.

Table 3. The conforming decimal formats. Each packs a sign, an exponent, and an integer significand into a word of the given width using the BID encoding.

Type	Storage	Digits p	Significand range
`decimal32_t`	32-bit	7	0 to $10^7 - 1$
`decimal64_t`	64-bit	16	0 to $10^{16} - 1$
`decimal128_t`	128-bit	34	0 to $10^{34} - 1$

3 Design and Architecture

3.1 Goals

The library is guided by three goals, in priority order: ease of use, portability, and performance. Ease of use means the types behave like the built-in floating-point types and integrate with the standard library, so that adopting decimal is largely a matter of changing a type name. Portability means a single source compiles and produces identical results on every supported target without per-platform code paths. Performance means decimal arithmetic should be competitive with, and where possible faster than, the established software implementations. This section describes the type set, the internal representation, and the `constexpr` strategy that ties the first two goals together; portability and performance are treated in Sections 6 and 9.

3.2 The decimal type set

Boost.Decimal provides two families of types. The conforming family, `decimal32_t`, `decimal64_t`, and `decimal128_t`, implements the IEEE 754 interchange formats exactly, including their bit-level encoding, so that a value can be stored, transmitted, and recovered across implementations. The second family, `decimal_fast32_t`, `decimal_fast64_t`, and `decimal_fast128_t`, computes the same numerical results but trades storage width for speed. Both families present the same interface, following the operator and function set proposed in ISO/IEC TR 24733 [15]. Mixed-type arithmetic follows rules in the spirit of the usual arithmetic conversions: an operation between two decimal types yields the higher-precision type, and an operation between a conforming type and its fast counterpart yields the fast type. Table 3 lists the three conforming formats and their parameters.

3.3 Internal representation

A conforming type stores a single unsigned integer holding the IEEE 754 BID encoding; `decimal32_t`, for instance, is a class wrapping one `std::uint32_t`. The sign, a combination field, the biased exponent, and the integer significand are all packed into those bits, so each operation must decode its operands, compute, and re-encode the result. A fast type instead stores the significand, exponent, and sign in three separate, already-normalized fields: `decimal_fast32_t`, for example, holds an unsigned significand, a small unsigned exponent, and a boolean sign. This removes both the per-operation decode and the normalization that the conforming types need in order to compare across cohorts (the several encodings of one value, such as 1e5 and 10e4, that decimal permits). The cost is that fast types occupy more memory, carry no cohort information, and flush subnormals to zero; the benefit is a measurable speedup, quantified in Section 9. The fast types are not, however, uniformly faster: producing the shortest output, for instance, requires stripping the trailing zeros that normalization introduces.

3.4 The constexpr strategy

A central decision is that the entire library is usable in a constant expression. C++14's relaxed `constexpr` rules make this possible: with loops and local mutation permitted in `constexpr` functions, arithmetic, parsing, formatting, and the elementary functions can all be evaluated at compile time, which lets the types stand in for built-in types in template metaprogramming and in contexts that require constant initialization. This choice recurs throughout the implementation. It precludes reliance on the C floating-point environment for exception flags (discussed below), and it shapes the code so that the same function bodies are valid both at compile time and on a GPU device, where the CUDA compiler eagerly type-checks and emits device code for every instantiation. The library detects whether an evaluation is occurring at compile time and selects the matching rounding-mode source, so that constant evaluation and run time agree.

Listing 2. Decimal arithmetic in a constant expression: the sum is formed and checked entirely at compile time.

```
using boost::decimal::decimal64_t;
constexpr decimal64_t tenth {1, -1};                          // 0.1
static_assert(tenth + tenth + tenth == decimal64_t{3, -1});   // 0.3, exactly
```

3.5 Standard-library integration and deviations

The types are accompanied by decimal versions of the standard headers a numeric type is expected to support, including `<cmath>`, `<charconv>`, `<cstdlib>`, `<cfenv>`, `<limits>`, `<format>`, and the user-defined literals. Meeting the C++14 baseline and the conceptual requirements of Boost.Math additionally makes the types usable with that library's statistics, quadrature, and interpolation facilities without extra work. A few deliberate deviations follow from the design goals and are documented as such. Because the library chooses `constexpr` over the C floating-point environment, it does not set or read the floating-point exception flags, which would otherwise require the C-style out-parameter signatures used by the Intel library. The elementary functions are not claimed to be correctly rounded to half a unit in the last place, matching both the behavior and the stated position of the reference C libraries. Input and output add a `cohort_preserving_scientific` format so that a value's cohort can be printed, `from_chars` reports overflow and underflow distinctly (following the resolution direction of LWG issue 3081) and honors the active rounding mode, and parsing of non-finite values from a stream is permitted. We return to the I/O deviations in Section 5.

4 Arithmetic Implementation

Every finite decimal value is a signed integer significand scaled by a power of ten. The four basic operations decode their operands into this significand, exponent, and sign form, compute an exact intermediate result in an integer wide enough to hold it, and then pass through a shared finalization step that shrinks the intermediate to the type's precision p (7, 16, or 34 digits for the three sizes), rounds it, and packs the value back into the result type. Finalization is also where a result that is too large becomes an infinity and one that is too small flushes to zero. For results known to be in range the finalizer writes the BID bit pattern directly and skips the general constructor's bounds checks; out-of-range results take the constructor path.

4.1 Addition and subtraction

Addition first aligns the operands to a common exponent by scaling the significand with the smaller exponent up by the appropriate power of ten, then adds or subtracts the aligned significands according to their signs. The result can exceed the target precision by a few digits, or, after a large

alignment shift, by many. The common case of one to three excess digits is detected by a chained comparison against precomputed thresholds, while the rare larger case is handled by counting digits and looking up the divisor in a power-of-ten table. Either way the excess is removed by a single division with round half to even; division by a constant power of ten is performed by reciprocal multiplication by an invariant divisor [10, 25] rather than a hardware divide. Subtraction is addition with the sign of the second operand inverted.

4.2 Multiplication

Multiplication adds the operand exponents and multiplies the significands by the schoolbook method, Knuth's Algorithm M [18]. With both significands expanded to full precision p, the exact product has $2p-1$ or $2p$ digits and is held in an integer twice the significand width: a 64-bit product for the 32-bit types, a 128-bit product for the 64-bit types, and a 256-bit product for the 128-bit types. The finalizer branches on the threshold 10^{2p-1} to decide whether to drop $p-1$ or p digits, divides by the corresponding power of ten with round half to even, handles the carry that arises when rounding lifts the quotient to 10^p, and packs the result.

4.3 Division

Division subtracts the operand exponents and forms the quotient by long division of the significands (Knuth's Algorithm D [18]), with the inner quotient-digit estimate using the reciprocal 3/2 method of Moller and Granlund [21]. The dividend significand is pre-scaled by 10^p so that dividing by the divisor significand yields a quotient of exactly p or $p+1$ digits together with a remainder, and that remainder is precisely the sticky information needed to round correctly. With both operands at full precision the quotient lies in $[10^{p-1}, 10^{p+1})$; the finalizer applies round half to even using the remainder, and the dropped low digit when the quotient has $p+1$ digits, again handling the carry to 10^p. The 32-bit case keeps the scaled dividend in a 64-bit integer, with the wider backend types used for the larger precisions.

4.4 Rounding and special values

The default rounding direction is round to nearest, ties to even, which the inline finalizers above assume; the other IEEE 754 rounding directions are honored by routing through a separate step that reads the active mode. Because the library is `constexpr`, it distinguishes a compile-time evaluation, where the rounding mode is a translation-time constant, from a run-time evaluation, where the mode is read through the `<cfenv>` interface, so that the two agree. Signed zeros, infinities, and quiet and signaling NaNs follow the IEEE 754 rules; for example, subtracting two like-signed infinities and multiplying zero by an infinity each produce a quiet NaN. Comparison of the conforming types normalizes cohorts so that numerically equal values compare equal regardless of encoding, whereas the fast types, being stored normalized, compare directly.

4.5 The wide-integer backend

The arithmetic above relies on integers wider than 64 bits. Rather than depend on a compiler's `__int128`, which is absent on 32-bit targets and on MSVC, the library bundles a self-contained backend providing 128-bit signed and unsigned types and a 256-bit type, with the multiplication and division primitives they require; on targets that offer a native 128-bit type the backend uses it. This single backend serves the wide products of multiplication, the scaled dividends of division, and the square-root computation below, and it is a principal reason one source compiles identically on every supported architecture (Section 6).

Table 4. Square root by precision: the integer width of the working arithmetic, the number of Newton-Raphson refinement iterations, and the final rounding rule.

Type	Integer width	Newton iterations	Final rounding
`decimal32_t`	64-bit	1	floor
`decimal64_t`	128-bit	2	floor
`decimal128_t`	256-bit	3	round to nearest

4.6 Square root

Square root illustrates how an elementary function is implemented entirely in integer arithmetic, so that it is usable in a constant expression and produces identical results on every target, given that no hardware decimal square root exists. The method follows Berkeley SoftFloat [11], adapted from radix two to radix ten. The argument is reduced to a value $g \in [1, 10)$ while its exponent is tracked separately; an initial approximation of $1/\sqrt{g}$ is read from a 90-entry table that covers $[1, 10)$ in steps of 0.1, with linear interpolation between adjacent entries; the approximation is refined by Newton-Raphson iterations expressed as integer remainder corrections of the form $z' = z + (x - z^2)/(2z)$; and the result is rescaled by the appropriate power of ten, with a further factor of $\sqrt{10}$ when the original exponent is odd. The number of refinement iterations and the integer width grow with precision, as Table 4 shows: the 128-bit case uses the 256-bit backend for the squaring and the remainder and rounds to nearest, while the smaller types round by floor. Recasting the computation in this integer form made it several times faster than the library's earlier square root, by roughly a factor of six to eight for the 32- and 64-bit types and about four for the 128-bit type.

4.7 Special functions

Beyond the basic operations, the library supplies a `constexpr <cmath>` surface that goes well past the standard's mandate, including the exponential, logarithmic, and power functions, the trigonometric and hyperbolic functions, the error and gamma family, and several orthogonal-polynomial and elliptic-integral families. The implementations use textbook methods adapted to fixed decimal precision: Cody-Waite range reduction [4] to bring an argument into a small interval, minimax (Remez) polynomial approximations [22] on that interval, the arithmetic-geometric mean for the complete elliptic integrals [19, 24], an accelerated alternating series for the Riemann zeta function [3], and Kahan compensated summation [16] where a long series would otherwise lose accuracy. They build on the multiple-precision special-function techniques of Algorithm 910 [19], by one of the present authors. Like the reference libraries they are not correctly rounded (Section 3); we do not count them among the contributions of this paper, but they show that the types form a complete numeric facility rather than a minimal arithmetic core.

5 Exact Decimal I/O and Conformant Conversion

For a decimal type the boundary between text and value is not an afterthought; it is the main reason to use the type. A price such as 19.99, entered as text, must become the value nineteen and ninety-nine hundredths exactly, and must print back as 19.99, with no binary rounding introduced on the way in or out. This section describes how Boost.Decimal parses and prints values, how it exposes the IEEE 754 encodings, and where its conversion interface deviates, by design, from the C++ standard.

5.1 Shortest round-trip output

The default text output produces the shortest decimal string that reads back to the same value, using the Ryu algorithm of Adams [1] generalized to the 128-bit significand of `decimal128_t`. Because the stored value is already base ten, the shortest-representation problem is simpler than it is for binary types, and the printed digits are exact rather than the nearest binary approximation. The conversion of the significand digits to text uses the JEAIII integer-to-string method [2]. Parsing and printing are offered through a `<charconv>`-style interface, `from_chars` and `to_chars`, which returns a pointer and a `std::errc` status and, like the rest of the library, is usable in a constant expression. A companion compile-time trait reports the largest buffer each format and precision can require, so callers can size output buffers exactly instead of guessing.

5.2 Formats and cohorts

The interface provides the scientific, fixed, hexadecimal, and general formats of the standard, plus one addition specific to decimal: a cohort-preserving scientific format. A cohort is the set of distinct encodings of one value that decimal admits; in a seven-digit type the value three hundred can be written as any of 3e+2 through 3000000e-4, each a different bit pattern denoting the same number. By default the conversion functions ignore cohorts and emit the normalized form, which is the behavior users expect from a numeric type. When a caller needs to record exactly which member of the cohort a value is, the cohort-preserving format reads and writes the significand and exponent verbatim, so that the round trip is bitwise and not merely numerically faithful. The format carries the constraints the standard implies: it is unavailable for the fast types, which store no cohort, and combining it with an explicit output precision is rejected, because rounding or zero-padding would destroy the very cohort information the format exists to preserve.

5.3 The IEEE 754 bit encodings

Decimal values can also be exchanged at the bit level. The library exposes conversions to and from both IEEE 754 significand encodings: the Binary Integer Decimal (BID) form it uses internally, and the Densely Packed Decimal (DPD) form used by hardware decimal units. The `to_bid`, `from_bid`, `to_dpd`, and `from_dpd` functions move a value to or from a fixed-width unsigned integer of the matching size, so a value produced by Boost.Decimal can be handed to hardware or to another implementation and recovered exactly. These conversions are likewise `constexpr` and run on a GPU device.

5.4 Conformance and deliberate deviations

The `from_chars` and `to_chars` interfaces follow the standard `<charconv>` functions where possible, and the library provides overloads taking `std::chars_format` that return the standard result types, so existing code ports with little change. Three deviations are intentional and follow from the nature of decimal. First, `from_chars` distinguishes overflow from underflow, by setting the output value, where the standard reports only a single out-of-range error; this follows the direction of LWG issue 3081 and matches Boost.Charconv. Second, because the decimal types are far more sensitive to rounding direction than binary types are, `from_chars` rounds according to the active rounding mode rather than always to nearest. Third, the cohort-preserving format above has no counterpart in the standard. Each deviation is documented, and the standard-compatible overloads remain available for code that requires strict conformance.

Table 5. Portability coverage. The library is built and tested across these axes; PPC64LE and the bare-metal target are exercised under QEMU emulation.

Dimension	Coverage
Operating systems	Linux, macOS, Windows
Architectures	x86-32, x86-64, ARM32, ARM64, s390x, PPC64LE
Word size	32-bit and 64-bit
Byte order	little-endian and big-endian (s390x)
Bare metal	ARM Cortex-M (no operating system)
Compilers	GCC 8+, Clang 6+, Visual Studio 2019+, Intel oneAPI

6 Portability and Cross-Platform Consistency

Portability in Boost.Decimal means more than compiling on many systems: it means that the same program produces the same bits on every one of them. This section describes how that is achieved and how it is verified.

6.1 One source, no per-platform arithmetic

The arithmetic is written once, in portable C++, and does not branch on the target architecture. Two mechanisms make this possible. The first is the bundled wide-integer backend of Section 4: where a native 128-bit integer exists the library uses it, and where it does not, on 32-bit targets and on MSVC, the emulated type takes over with identical behavior, so multiplication and division need no architecture-specific code. The second is explicit control of byte order. Because a conforming value is a packed bit pattern, its bytes must be interpreted consistently regardless of the host's endianness; the library determines endianness at compile time and refuses to build, with a request to file an issue, on any target whose byte order it cannot establish, rather than silently producing wrong results. This is what lets one BID encoding be correct on little-endian x86 and on big-endian s390x alike.

6.2 Tolerating compiler and standard-version differences

A header-only library that supports compilers as old as GCC 8 and Clang 6 must absorb a wide range of compiler behavior without changing its results. Boost.Decimal does so by detecting language and library features and degrading gracefully: it uses `if constexpr` where available and a plain conditional under C++14, a `constexpr` bit-cast where the compiler supports one and a fallback where that support is known to be broken (for instance on a particular Clang release and on 32-bit targets), and a working detection of constant evaluation across the several compiler versions that implement it differently. The C++14 floor is itself a portability choice: it is the lowest standard on which the `constexpr` design can be expressed, and it admits the largest set of toolchains.

6.3 The tested matrix

The library is built and tested across the axes listed in Table 5, spanning 32- and 64-bit word sizes, little- and big-endian byte orders, and hosted and bare-metal targets, with the major compiler on each. Continuous integration builds and runs the full test suite, including the conformance suite of Section 8, on this matrix on every change, and the tests assert identical numerical results across it, so a regression on any one architecture, endianness, or word size is caught at once. It is this cross-platform agreement, verified rather than assumed, that makes the library suitable where reproducibility across machines is a requirement.

Table 6. Operations validated on the CUDA device, by precision. Each device result is checked against the host result. Mixed-precision combinations of the three types are also exercised.

Operation	`decimal32_t`	`decimal64_t`	`decimal128_t`
Construction	yes	yes	yes
Add, subtract, multiply, divide	yes	yes	yes
Comparison	yes	yes	yes
BID encode and decode	yes	yes	yes
DPD encode and decode	yes	yes	yes
Mathematical constants	yes	yes	yes
Text conversion (charconv)	yes	yes	yes

7 GPU and Accelerator Support

A distinguishing capability of Boost.Decimal is that the same decimal types run on an NVIDIA GPU. Because the established libraries are host C code, decimal arithmetic has not previously been available inside CUDA kernels. This section describes how the library provides it and what is tested.

7.1 One source for host and device

The `constexpr` discipline of Section 3 is what makes device support practical. Every function the library exposes is annotated with a macro that expands to nothing for an ordinary host build and to the CUDA `__host__ __device__` qualifiers when the translation unit is compiled by the CUDA compiler with device support enabled. Because the function bodies were already written to be valid in a constant expression, they contain no constructs, no exceptions and no run-time-only library calls, that would prevent the device compiler from accepting and emitting them, so a single source serves both host and device with no separate device implementation to maintain.

7.2 Device execution

On the device the library makes a few automatic adjustments that the host build does not require: it uses the bundled emulated 128-bit integer rather than a compiler built-in, since device support for native 128-bit integers is limited, and it disables exceptions and the assertion macro, which have no place in device code. The numerically meaningful behavior is unchanged, so a decimal computation performed in a kernel produces the same result as the same computation on the host. The arithmetic, the comparisons, the BID and DPD conversions, the mathematical constants, and text conversion through `<charconv>` are all available on the device, across the three precisions and in mixed-precision combinations.

7.3 Testing on the device

Device support is validated by a dedicated CUDA test suite. For each of `decimal32_t`, `decimal64_t`, and `decimal128_t`, and for mixed combinations of them, kernels exercise construction, the four arithmetic operations, comparison, the BID and DPD encodings, the constants, and `<charconv>`; inputs are transferred and results read back through CUDA managed memory, and each device result is checked against the host computation. Passing this suite is the evidence behind the claim that the library computes identically on the GPU and on the CPU. Table 6 summarizes what the device suite covers.

7.4 Scope of the novelty claim

We are careful about the scope of this claim. To our knowledge no prior IEEE 754 decimal floating-point library runs on a GPU. The nearest existing capability, the decimal column types in NVIDIA's

Table 7. Operations covered by the vendored General Decimal Arithmetic conformance vectors. The vectors are supplied as the 64-bit (dd) and 128-bit (dq) test sets, with a subset for the 32-bit format, in 35 files in total.

Category	Operations
Arithmetic	add, subtract, multiply, divide
Comparison	compare, compare-signaling, compare-total-order
Rounding	quantize, remainder
Other	absolute value, negate, maximum, minimum, clamp

cuDF, is fixed-point rather than floating-point (Section 2), and the hardware's own IEEE 754 support is binary. We therefore state the contribution as the first decimal floating-point library with GPU support that we are aware of, rather than as an absolute first, and we note that the library does not at present provide a non-CUDA accelerator path such as SYCL.

8 Conformance and Verification

Correctness is the first requirement of a numerical library, and for a portable one it must hold identically on every target. Boost.Decimal is checked at four levels: against the official decimal arithmetic conformance suite, by property-based and fuzz testing of the most input-exposed code, by a body of regression tests, and by the cross-platform agreement of Section 6. Together these form the evidence on which a Replicated Computational Results review (Appendix A) would rest.

8.1 Conformance to IEEE 754

The conforming types implement the IEEE 754 decimal formats, their arithmetic, the rounding directions, and the special values. Two deviations are documented and deliberate, both discussed in Section 3: the library does not maintain the floating-point exception flags, having chosen compile-time evaluation over the C floating-point environment, and the elementary functions are not claimed to be correctly rounded to half a unit in the last place, which matches the behavior and the stated position of the reference C libraries. The mandated operations, including the basic arithmetic and square root, are otherwise implemented to the standard's requirements.

8.2 The decimal conformance suite

The primary correctness check is the General Decimal Arithmetic test suite, the machine-readable conformance vectors that accompany Cowlishaw's specification [6] and against which the reference implementations are themselves tested. The suite is vendored into the repository and driven by a parser and runner that reads each vector, performs the named operation in the appropriate type, and compares the result and status against the expected values. The vectors cover the core operation set, including absolute value, addition, subtraction, multiplication, division, the comparison operations and their signaling and total-order variants, maximum and minimum, quantize, and remainder, in both the 64-bit and 128-bit formats. Table 7 groups the covered operations. Running the same external suite that the Intel and IBM libraries use is what lets us claim conformance rather than merely internal consistency.

8.3 Property-based and fuzz testing

The text-conversion surface is the most exposed to untrusted input and the most prone to edge cases, so it is fuzzed continuously. A set of libFuzzer harnesses drives parsing and formatting through every entry point, including `from_chars`, `strtod`-style parsing, the string constructors, and the `printf`, `<format>`, and {fmt}-style output paths, each seeded with a corpus and run in

continuous integration. Round-trip properties, that printing a value and reading it back recovers the value, and that the cohort-preserving format recovers the exact bit pattern, are checked directly.

8.4 Regression and cross-platform checks

Every bug fixed during development is captured as a regression test so that the same defect cannot reappear; there are at present roughly forty such tests, alongside the operation- and function-level unit tests. Crucially, the entire suite, the conformance vectors, the property tests, and the regression tests, is run on every target in the portability matrix of Table 5, and the results are required to agree bit for bit. Line coverage is measured and reported in continuous integration. A defect that appeared only on a big-endian or a 32-bit target, or only during constant evaluation, would be caught by this matrix rather than discovered in the field.

9 Performance Evaluation

This section quantifies how Boost.Decimal compares with the established software implementations of decimal arithmetic. The headline question is not whether decimal is slower than binary, it is, because binary floating-point has hardware support on every target of interest while all decimal arithmetic here is software, but whether Boost.Decimal is competitive with the reference decimal software: the Intel library and the GCC built-in types.

9.1 Methodology

Each arithmetic benchmark fills a vector of twenty million random values and applies the operation between successive elements, repeating the measurement five times to obtain a stable figure; the comparison benchmark applies the six relational operators, and the text benchmarks perform twenty million conversions. The same harness measures the built-in `double`, the Boost.Decimal conforming and fast types, the GCC `_Decimal` built-ins, and the Intel BID library. One caveat applies throughout: the Intel and GCC types are driven through C routines that are close to, but not identical with, the C++ routines used for the Boost.Decimal types, because those types are C constructs; we take the routines to be near enough for a fair comparison, as their authors' own benchmarks do. The complete per-operation and per-platform tables are in the library documentation; we report a representative subset here.

9.2 Platforms

Results were collected on four configurations: x86-64 Linux on an Intel i9-11900k with both the Intel oneAPI compiler and GCC 13.4; x86-64 Windows on the same processor with Visual Studio; ARM64 macOS on an Apple M4 Max with Clang; and ARM64 Windows. The trends below hold across all four; we draw the table from the x86-64 Linux GCC configuration because it measures every baseline at once.

9.3 Results

Table 8 reports the time to perform twenty million 64-bit operations on x86-64 Linux. Against the Intel reference library, Boost.Decimal is faster on every operation, by factors ranging from roughly two for multiplication to about ten for comparison, and the fast type widens the margin further. Against the GCC `_Decimal64` built-in, the fast type is faster for addition, subtraction, and division, and comparable for multiplication and comparison. The conforming `decimal64_t`, which additionally pays for cohort handling, is competitive with the GCC type and remains well ahead of the Intel library.

The advantage is largest at 128 bits, where the reference library is weakest: on x86-64 Linux with the Intel compiler, multiplying `decimal_fast128_t` is about 3.7 times faster than the Intel

Table 8. Time in microseconds to perform twenty million 64-bit operations on x86-64 Linux (Intel i9-11900k, GCC 13.4); lower is better. All four columns are software implementations of decimal arithmetic. For reference, hardware binary double completes the same arithmetic workloads in well under one hundred thousand microseconds.

Operation	decimal64_t	decimal_fast64_t	GCC _Decimal64	Intel BID_UINT64
Comparison	1,090,026	513,587	475,605	5,452,325
Addition	1,257,515	1,064,025	2,100,094	3,379,104
Subtraction	1,313,266	1,166,909	1,313,261	3,522,244
Multiplication	2,528,577	2,312,590	2,462,813	4,973,224
Division	2,477,436	1,724,535	2,904,760	5,745,255

Table 9. Time in microseconds to perform twenty million 128-bit operations on x86-64 Linux (Intel i9-11900k, GCC 13.4); lower is better. All four columns are software implementations of decimal arithmetic.

Operation	decimal128_t	decimal_fast128_t	GCC _Decimal128	Intel BID_UINT128
Comparison	3,772,309	444,145	872,026	5,705,342
Addition	2,085,250	926,772	3,264,420	2,509,923
Multiplication	7,966,781	8,116,908	6,683,052	21,352,000
Division	5,854,061	4,442,123	9,906,049	14,918,326

BID_UINT128, and on Windows the multiplication gap is wider still. With the Intel compiler the fast 64-bit addition is about 4.6 times faster than BID_UINT64, somewhat better than the GCC-toolchain figure in the table, which reflects the compiler difference rather than a change in the library. Table 9 gives the 128-bit operations under the GCC toolchain. It also shows the one case where a competitor leads: at 128-bit multiplication the GCC built-in is modestly ahead of Boost.Decimal, though both remain far ahead of the Intel library.

For text conversion the picture changes, because binary <charconv> is also software: here the decimal types are competitive with or faster than double. On x86-64 Linux, formatting at a fixed six-digit precision, decimal32_t and decimal64_t cost roughly half as much as double in scientific notation and well under it in general notation, and parsing is within a small factor of double across the formats.

9.4 Summary

Across the basic operations and on every platform measured, Boost.Decimal is faster than the Intel reference library, and it is faster than the GCC built-in decimal types in many operations though not all; for text conversion it is competitive with or faster than hardware-backed binary double. We state the result deliberately as parity or improvement over the reference software, not as a universal speed record: against hardware binary floating-point, decimal arithmetic remains several times slower, the expected cost of exact base-ten behavior.

10 Applications: Composability with Boost.Math

The applications payoff of Boost.Decimal is not that it reproduces base-ten arithmetic; that follows from the design and was shown in Sections 5 and 6. It is that, because the types are standards-conforming C++ types rather than an opaque C library, they compose with the existing C++ numerical ecosystem. A large body of analysis becomes available on exact decimal data with no new code, and with the same cross-platform reproducibility as the arithmetic. This is something

the Intel and IBM C libraries cannot offer, because they are not C++ types and do not model the concepts that generic numerical code is written against.

10.1 Exact input

We use a year of daily stock prices as a running dataset. Read with the decimal types, the data is exact: summing 252 closing prices in `decimal32_t` reproduces the spreadsheet total of 52151.99 to the cent, whereas the same sum in `float` accumulates to 52151.96. The error is not in the addition but in the representation, and it is removed at the point of parsing. With the values exact, any subsequent analysis starts from the right numbers.

10.2 Statistics for free

Because the decimal types satisfy the user-defined-type requirements of Boost.Math, that library's algorithms operate on a `std::vector` of decimals directly. The same calls one would write for `double`, the mean, median, and variance functions of `boost::math::statistics`, compute the closing-price statistics, and a square root from the library's own `<cmath>` gives the standard deviation, from which two-sigma Bollinger bands follow. On the sample year this yields a mean closing price of $207.20, a median of $214.26, a standard deviation of $25.45, and bands at $258.11 and $156.30. No decimal-specific statistics code was written: the analysis is the generic Boost.Math algorithm instantiated on the decimal type. In practice a user enables one documented macro so that the mixed integer-decimal expressions inside the generic code compile, and suppresses a few of Boost.Math's float-comparison warnings; the algorithms themselves are unchanged.

Listing 3. Boost.Math's statistics algorithms run on a vector of decimals with no decimal-specific code; the Bollinger band follows from the mean and the standard deviation.

```
using boost::decimal::decimal64_t;
namespace stats = boost::math::statistics;

std::vector<decimal64_t> closes = /* parsed exactly from the CSV */;
const decimal64_t mean = stats::mean(closes);
const decimal64_t var  = stats::variance(closes);
const decimal64_t sd   = boost::decimal::sqrt(var);
const decimal64_t upper = mean + 2 * sd;   // upper two-sigma Bollinger band
```

10.3 The breadth, and why it comes for free

The statistics above are one corner of what the same concept conformance unlocks. The library's continuous integration exercises the decimal types with Boost.Math across a much wider surface: the univariate statistics suite in full (the first four moments, variance and sample variance, skewness, kurtosis, median and median absolute deviation, the interquartile range, the Gini coefficient, and the mode), and a range of special functions (the beta and Riemann zeta functions, the elliptic integrals, and the Legendre, Laguerre, and Hermite polynomial families), with the library's own elementary functions cross-checked against Boost.Math's. The rest of the Boost.Math toolkit, its statistical distributions, quadrature, interpolation, and root-finding, is reachable through the same mechanism, since all of it is generic code written against the same numeric concepts.

The reason this comes essentially for free is generic programming. A Boost.Math algorithm is written once against a concept and instantiated on any type that models it; meeting the C++14 baseline and those concept requirements (Section 3) is therefore sufficient to make the toolkit available on decimal values, and the results inherit the bit-for-bit cross-platform consistency of Section 6. A C library of decimal routines, however complete, cannot be dropped into this generic

code, because it provides functions rather than a type that models the concepts. The composability is thus not an add-on but a direct consequence of implementing decimal as a first-class C++ type, and it is, for a numerical-software audience, among the most consequential practical results of the design.

11 Discussion and Limitations

The design choices behind Boost.Decimal carry costs as well as benefits, and an honest account of them sharpens the contribution.

11.1 The conforming and fast types

The fast types buy speed with space and standards conformance. They are wider than the conforming types, they do not represent cohorts (so they cannot preserve a value's exact encoding through I/O), and they flush subnormal values to zero. They are also not uniformly faster: producing the shortest decimal string, for instance, requires removing the trailing zeros that their normalized storage introduces, which can make text output slower than for the conforming types. A user therefore chooses per use: the conforming types where the IEEE 754 bit layout, cohorts, or subnormals matter, and the fast types where throughput dominates.

11.2 The cost relative to binary

Decimal arithmetic in Boost.Decimal is software, and on every platform of interest binary floating-point is hardware; the basic operations are accordingly several times to a few tens of times slower than `double` (Section 9). This is inherent to decimal rather than to this implementation, and it bounds the library's applicability: it is the right tool where exact base-ten behavior or cross-platform reproducibility is required, and the wrong one for bulk numerical work that is indifferent to base. The elementary functions are likewise not correctly rounded to half a unit in the last place, matching the reference C libraries but falling short of a correctly-rounded `libm`.

11.3 Accelerator and platform scope

GPU support is at present specific to CUDA; there is no SYCL or other portable accelerator backend, and on the device the library uses its emulated 128-bit integer because native support is limited. The tested platform matrix is broad but not exhaustive, and the cross-platform consistency guarantee is only as strong as that matrix.

11.4 Threats to validity in the comparison

The performance comparison has the limitations its methodology implies. The Intel and GCC baselines are driven through C routines that are close to, but not identical with, the C++ routines used for the decimal types, so small differences may reflect the harness rather than the arithmetic. Results were collected on a few machines rather than averaged over many, and they depend on the compiler and its optimization settings, as the gap between the Intel-compiler and GCC figures in Section 9 shows. We therefore present the comparison as evidence of parity or better with the reference software, not as a precise speedup factor.

12 Conclusions and Future Work

Boost.Decimal shows that a single, portable, header-only C++ source can provide IEEE 754 decimal floating-point that runs at compile time, on the CPU, on a GPU, and on bare-metal targets, with results identical across architectures and performance that matches or exceeds the established reference software. The formats and the encoding are not new; the contribution is to deliver them together, as a first-class C++ type, without the integration cost, the loss of `constexpr`, or the

platform limits of the existing C libraries. Because the type models the concepts that generic numerical code expects, the broader C++ ecosystem, Boost.Math among it, operates on exact decimal data unchanged, which turns a single arithmetic type into a foundation for reproducible decimal analysis.

Several directions remain. A correctly-rounded mode for the elementary functions would close the one conformance gap the library shares with the reference C libraries. A portable accelerator backend, such as SYCL, would extend device support beyond CUDA. The tested platform matrix can be broadened further, and the Boost.Math composability of Section 10 invites larger case studies in the domains, finance and commerce above all, where exact decimal analysis matters most. The library is released under the Boost Software License and developed in the open.

Acknowledgments

We thank The C++ Alliance for sponsoring the development of this work, and the Boost community reviewers and the review manager, John Maddock, for the peer review that preceded the library's acceptance into Boost.

References

[1] Ulf Adams. 2018. Ryu: Fast Float-to-String Conversion. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. 270–282. doi:10.1145/3192366.3192369

[2] James Edward III Anhalt. [n. d.]. The JEAIII Integer-to-String Algorithm. https://jk-jeon.github.io/posts/2022/02/jeaiii-algorithm/. Online writeup by Junekey Jeon. Verify canonical citation before camera-ready.

[3] Peter Borwein. 1995. An Efficient Algorithm for the Riemann Zeta Function. Accelerated alternating series. Verify exact venue before camera-ready.

[4] William J. Cody and William Waite. 1980. *Software Manual for the Elementary Functions*. Prentice-Hall.

[5] Marius Cornea, John Harrison, Cristina Anderson, Ping Tak Peter Tang, Eric Schneider, and Evgeny Gvozdev. 2009. A Software Implementation of the IEEE 754R Decimal Floating-Point Arithmetic Using the Binary Encoding Format. *IEEE Trans. Comput.* 58, 2 (2009), 148–162. doi:10.1109/TC.2008.209

[6] Michael F. Cowlishaw. 2003. Decimal Floating-Point: Algorism for Computers. In *Proceedings of the 16th IEEE Symposium on Computer Arithmetic (ARITH-16)*. doi:10.1109/ARITH.2003.1207666

[7] Michael F. Cowlishaw and IBM Corporation. 2010. The decNumber Library and General Decimal Arithmetic. https://speleotrove.com/decimal/. Portable ANSI C reference implementation; used by GCC and ICU.

[8] Goran Flegar, Florian Scheidegger, Vedran Novaković, Giovanni Mariani, Andrés E. Tomás, A. Cristiano I. Malossi, and Enrique S. Quintana-Ortí. 2019. FloatX: A C++ Library for Customized Floating-Point Arithmetic. *ACM Trans. Math. Software* 45, 4, Article 40 (2019). doi:10.1145/3368086

[9] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélissier, and Paul Zimmermann. 2007. MPFR: A Multiple-Precision Binary Floating-Point Library with Correct Rounding. *ACM Trans. Math. Software* 33, 2, Article 13 (2007). doi:10.1145/1236463.1236468

[10] Torbjörn Granlund and Peter L. Montgomery. 1994. Division by Invariant Integers using Multiplication. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (PLDI)*. 61–72. doi:10.1145/178243.178249

[11] John R. Hauser. 2018. Berkeley SoftFloat, Release 3e. http://www.jhauser.us/arithmetic/SoftFloat.html.

[12] IEEE. 2008. IEEE Standard for Floating-Point Arithmetic. *IEEE Std 754-2008* (2008), 1–70. doi:10.1109/IEEESTD.2008.4610935

[13] IEEE. 2019. IEEE Standard for Floating-Point Arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)* (2019), 1–84. doi:10.1109/IEEESTD.2019.8766229

[14] Intel Corporation. 2023. Intel Decimal Floating-Point Math Library. https://www.intel.com/content/www/us/en/developer/articles/tool/intel-decimal-floating-point-math-library.html. Accessed 2026. Software implementation of IEEE 754-2008 decimal arithmetic.

[15] ISO/IEC. 2011. ISO/IEC TR 24733: Extension for the Programming Language C++ to Support Decimal Floating-Point Arithmetic. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2849.pdf.

[16] William M. Kahan. 1965. Pracniques: Further Remarks on Reducing Truncation Errors. *Commun. ACM* 8, 1 (1965), 40. doi:10.1145/363707.363723

[17] Robert Klarer et al. 2012. N3407: Proposal for Adding Decimal Floating Point Support to C++. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3407.html. WG21 paper.

[18] Donald E. Knuth. 1997. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley. Algorithm D (long division) and Algorithm M (multiplication), Section 4.3.1.
[19] Christopher Kormanyos. 2011. Algorithm 910: A Portable C++ Multiple-Precision System for Special-Function Calculations. *ACM Trans. Math. Software* 37, 4, Article 45 (2011). doi:10.1145/1916461.1916469
[20] Mark Mendell et al. 2014. N3871: Decimal Floating-Point Support. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3871.html. WG21 paper.
[21] Niels Möller and Torbjörn Granlund. 2011. Improved Division by Invariant Integers. *IEEE Trans. Comput.* 60, 2 (2011), 165–175. doi:10.1109/TC.2010.143
[22] Jean-Michel Muller. 2016. *Elementary Functions: Algorithms and Implementation* (3rd ed.). Birkhäuser. doi:10.1007/978-1-4899-7983-4
[23] NVIDIA Corporation. 2021. Implementing High-Precision Decimal Arithmetic with CUDA int128. https://developer.nvidia.com/blog/implementing-high-precision-decimal-arithmetic-with-cuda-int128/. cuDF GPU decimal types are fixed-point, not IEEE 754 floating-point.
[24] F. W. J. Olver et al. [n. d.]. NIST Digital Library of Mathematical Functions. https://dlmf.nist.gov/. Section 19.8, arithmetic-geometric mean for elliptic integrals.
[25] Henry S. Warren. 2013. *Hacker's Delight* (2nd ed.). Addison-Wesley.

A Reproducibility (for the RCR review)

All results in this paper can be reproduced from the public repository. This appendix lists what to build and how, so a reviewer can replicate the conformance results and the benchmarks.

Source. The results were produced from a tagged commit of the repository at https://github.com/boostorg/decimal; the exact tag is recorded with the artifact. The library is header-only and requires only a C++14 compiler.

Tests and conformance. The full test suite, including the General Decimal Arithmetic conformance vectors of Section 8, is built and run with Boost.Build from the library's `test` directory:

```
b2 cxxstd=20 toolset=gcc-13
```

Running the same command under other toolsets (for example `toolset=clang` or `toolset=msvc`) and on the architectures of Table 5 reproduces the cross-platform agreement; targets without native hardware are exercised under QEMU.

Benchmarks. The benchmarks are compiled into the test programs and enabled with a preprocessor definition. From the `test` directory:

```
b2 cxxstd=20 toolset=gcc-13 \
   define=BOOST_DECIMAL_RUN_BENCHMARKS benchmarks -a release
```

Appending `,BOOST_DECIMAL_BENCHMARK_CHARCONV=1` to the definition also runs the text-conversion benchmarks. The reference baselines build directly from their C sources: the GCC built-in decimal baseline with

```
gcc benchmark_libdfp.c -O3 -std=c17
```

and the Intel BID baseline by linking the Intel library,

```
icx benchmark_libbid.c -O3 $PATH_TO_LIBBID/libbid.a -std=c17
```

each producing an executable that prints per-operation timings.

Hardware and reading the results. The figures in Section 9 were collected on an Intel i9-11900k for the x86-64 results and an Apple M4 Max for the ARM64 macOS results. Each benchmark reports the total time in microseconds for twenty million operations, repeated five times for stability, so lower numbers are better, and the ratios between types are the quantities of interest rather than the absolute times.